

СОДЕРЖАНИЕ

Введение.....	6
1 Обзор литературы	7
1.1 Многоагентные системы на основе искусственного интеллекта	7
1.2 LangChain	8
1.3 LangGraph	9
1.4 Архитектурный паттерн Retrieval-Augmented Generation	11
1.5 Модели для семантического анализа и формирования векторных представлений	12
1.6 Векторные базы данных для хранения и поиска	14
1.7 Обзор аналогов	16
1.8 Технологии парсинга и обработки документов	19
2 Системное проектирование	21
2.1 Блок взаимодействия с пользователем	21
2.2 Блок обеспечения безопасности и контроля доступа	22
2.3 Блок агрегации и первичной обработки данных	22
2.4 Блок парсинга и извлечения сущностей	22
2.5 Блок генерации векторных представлений	23
2.6 Блок векторного хранилища и индексации	23
2.7 Блок гибридного поиска и ранжирования	23
2.8 Блок оркестрации и координации агентов	24
2.9 Общие принципы взаимодействия блоков	24
3 Функциональное проектирование	25
3.1 Описание сервиса extraction_service	25
3.2 Описание сервиса storage_service	27
3.3 Описание сервиса search_service	28
3.4 Описание сервиса orchestrator_service	29
3.5 Описание сервиса frontend	32
3.6 Описание сервиса auth_service	33
4 Разработка Программных модулей	35
4.1 Текст	35
5 Программа и методика испытаний	36
5.1 Подраздел	36
6 Руководство пользователя	37
6.1 Подраздел	37
7 Техничко-экономическое обоснование разработки системы подбора персонала с семантическим анализом агрегированных данных	38
7.1 Характеристика программного средства, разрабатываемого для реализации на рынке	38
7.2 Расчет инвестиций в разработку программного средства для его реализации на рынке	38
7.3 Расчет инвестиций в разработку программного средства для его реализации на рынке	41

7.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке.....	43
7.5 Вывод об экономической целесообразности реализации проектного решения.....	44
Заключение	45
Список использованных источников	46
Приложение А	48
Приложение Б	49
Приложение В.....	50
Приложение Г	51
Приложение Д.....	52

ВВЕДЕНИЕ

В условиях цифровой трансформации рынка труда и экспоненциального роста объемов данных о соискателях особую актуальность приобретает разработка интеллектуальных систем автоматизированного подбора персонала [1]. Современные рекрутинговые платформы, профессиональные сети и корпоративные базы данных формируют распределённые массивы информации о кандидатах, включающие резюме в различных форматах, профили в социальных сетях, результаты тестирований и рекомендательные письма.

Резюме и иные источники данных о кандидатах характеризуются высокой степенью неструктурированности, разнообразием лексических формулировок для описания одних и тех же компетенций (например, «machine learning engineer», «ML-разработчик», «специалист по машинному обучению»), а также наличием шумовых компонентов в виде маркетинговых формулировок и субъективных оценок. Традиционные системы поиска, основанные на ключевых словах и жёстких фильтрах, не способны учитывать семантическую близость понятий и контекстуальные связи между требованиями вакансии и опытом кандидата, что приводит к низкой релевантности результатов и увеличению трудозатрат рекрутеров [2]. Для повышения качества подбора персонала необходимо обеспечить агрегацию разнородных данных из множества источников с последующей их нормализацией и семантической интерпретацией.

Целью дипломного проектирования является разработка и реализация интеллектуальной системы подбора персонала с семантическим анализом агрегированных данных на основе многоагентной архитектуры, обеспечивающей автоматизированную обработку резюме, генерацию мультязычных эмбеддингов и гибридный поиск релевантных кандидатов.

В соответствии с целью определены следующие задачи:

- анализ существующих подходов к семантическому поиску в рекрутинге и выбор архитектурных решений;
- проектирование многоагентной системы агрегации и нормализации данных о кандидатах;
- реализация модуля извлечения структурированной информации из резюме в различных форматах;
- интеграция эмбеддинг модели для генерации семантических векторных представлений профилей кандидатов;
- разработка гибридного механизма поиска, комбинирующего семантическое сходство и полнотекстовые фильтры;
- создание пользовательского интерфейса для взаимодействия рекрутера с системой и валидация результатов поиска.

Данный дипломный проект выполнен мной лично, проверен на заимствования, процент оригинальности составляет XX% (отчет о проверке на заимствования прилагается).

1 ОБЗОР ЛИТЕРАТУРЫ

1.1 Многоагентные системы на основе искусственного интеллекта

1.1.1 Многоагентные системы представляют собой класс архитектур искусственного интеллекта, в которых для решения сложных задач взаимодействуют несколько автономных программных сущностей – агентов [3]. Агент в данном контексте определяется как самостоятельная вычислительная единица, обладающая способностью воспринимать окружающую среду (в виде входных данных), анализировать полученную информацию, принимать решения и выполнять действия, направленные на достижение поставленной цели. Каждый агент может быть специализирован на выполнении ограниченного круга функций, а их совместная работа позволяет решать задачи, превышающие возможности отдельного агента или традиционной монолитной программы.

В отличие от последовательных алгоритмов или простых цепочек обработки, многоагентные системы характеризуются распределённостью, возможностью динамического планирования и наличием механизмов координации. Агенты обмениваются информацией, делегируют задачи и корректируют поведение в зависимости от промежуточных результатов. Такой подход особенно эффективен при обработке больших объёмов неструктурированных данных, требующих последовательного анализа, синтеза и принятия решений. В области подбора персонала многоагентные системы применяются для автоматизации этапов сбора данных о кандидатах, извлечения ключевых характеристик, семантического сопоставления профилей с требованиями вакансий и формирования ранжированного списка рекомендаций.

1.1.2 Основой функционирования современных агентов служат большие языковые модели (large language models, LLM) [4]. Большая языковая модель – это нейронная сеть, обученная на огромных массивах текстовых данных, способная глубоко понимать естественный язык, генерировать связные и грамматически правильные тексты, выполнять логические рассуждения, отвечать на вопросы, переводить, суммировать, писать код и решать широкий спектр задач, связанных с обработкой и генерацией текста.

В основе архитектуры практически всех современных больших языковых моделей лежит трансформер – структура, предложенная в 2017 году в статье «Attention Is All You Need» [5]. Трансформер отказался от рекуррентных слоёв (RNN, LSTM), которые обрабатывали последовательности шаг за шагом, и ввёл механизм самовнимания (self-attention). Этот механизм позволяет модели одновременно учитывать взаимосвязи между всеми токенами в последовательности, независимо от расстояния между ними. Самовнимание вычисляет, насколько каждый токен важен для понимания других токенов в контексте, присваивая им веса внимания. Это даёт возможность эффективно захватывать долгосрочные

зависимости (например, связь между подлежащим и сказуемым в длинном предложении) и обрабатывать текст параллельно, что значительно ускоряет обучение и инференс по сравнению с предыдущими архитектурами.

1.1.3 К 2026 году большие языковые модели эволюционировали в сторону разреженных архитектур, среди которых ключевое место занимает Mixture-of-Experts (MoE, смесь экспертов) [6]. В отличие от традиционных плотных моделей, где все параметры активируются для обработки каждого токена, в MoE-моделях общее количество параметров может быть очень большим, но для каждого входного токена активируется лишь небольшая доля из них – обычно от нескольких миллиардов до десятков миллиардов, что резко снижает вычислительные затраты при сохранении высокого качества. Примером служит семейство Qwen3/Qwen3.5 от Alibaba, которое сочетает MoE с механизмами линейного внимания, поддерживает нативную мультимодальность и режимы глубокого рассуждения и быстрого ответа [7]. Такие модели демонстрируют конкурентные результаты на бенчмарках в задачах рассуждения, кодирования, математики и агентивного поведения, часто превосходя более крупные плотные модели при значительно меньшем потреблении ресурсов.

1.2 LangChain

LangChain представляет собой открытую программную платформу, предназначенную для разработки приложений на основе больших языковых моделей [8]. Созданная в 2022 году, платформа к 2026 году прошла несколько крупных обновлений и стала одним из наиболее широко используемых инструментов для построения сложных AI-систем. LangChain предлагает модульный подход к интеграции больших языковых моделей с внешними источниками данных, инструментами и механизмами управления состоянием, что позволяет создавать приложения, выходящие за рамки простого генеративного текста. Основная цель платформы – упростить переход от прототипирования к производственным решениям за счёт унифицированных интерфейсов, надёжных абстракций и инструментов наблюдения за поведением системы.

1.2.1 Принцип модульности LangChain позволяет конструировать приложения из независимых переиспользуемых блоков. Ключевые компоненты включают модели, обеспечивающие унифицированный доступ к различным большим языковым моделям, включая локальные и облачные реализации; шаблоны запросов, стандартизирующие формирование входных данных для моделей с поддержкой переменных, примеров и динамического контекста; цепочки обработки, представляющие собой последовательные комбинации вызовов моделей, преобразований данных и инструментов; инструменты, реализующие внешние функции, такие как доступ к базам данных, парсинг файлов, вызов API или математические вычисления;

механизмы памяти для хранения краткосрочного и долгосрочного контекста; ретриверы, отвечающие за извлечение релевантной информации из векторных хранилищ с поддержкой семантического, ключевого и гибридного поиска. Агент в рамках LangChain реализует цикл рассуждения и действий: анализирует задачу, планирует последовательность вызовов инструментов, выполняет их и корректирует поведение на основе полученных результатов. Компоненты экосистемы LangChain отображены на рисунке 1.1.



Рисунок 1.1 – Компоненты экосистемы LangChain

1.2.2 В системах подбора персонала агенты, реализованные на платформе LangChain, применяются для автоматизированного извлечения структурированной информации из резюме и сопроводительных документов с использованием цепочек парсинга, анализа профессиональных компетенций через семантическое сравнение текстовых описаний навыков и опыта; предварительной фильтрации кандидатов на основе гибридного поиска по векторным и атрибутивным критериям; формирования ранжированного списка рекомендаций с объяснением релевантности. Для сложных многоагентных сценариев необходимо комбинировать LangChain с LangGraph, что обеспечит явное управление состоянием, ветвлениями и устойчивостью к ошибкам.

1.3 LangGraph

1.3.1 LangGraph является расширением экосистемы LangChain, предназначенное для построения управляемых состоянием многоагентных и многошаговых процессов в форме направленных графов [9]. В отличие от линейных цепочек (chains) и относительно простых агентов в базовом LangChain, LangGraph позволяет явно моделировать сложные рабочие процессы как граф, где каждый узел соответствует действию, вызову модели, инструменту или человеческому вмешательству, а направленные рёбра задают возможные переходы, включая условные ветвления, циклы, возвраты к

предыдущим состояниям и параллельное выполнение. Визуальное сравнение систем, построенных на LangChain и LangGraph отображено на рисунке 1.2.

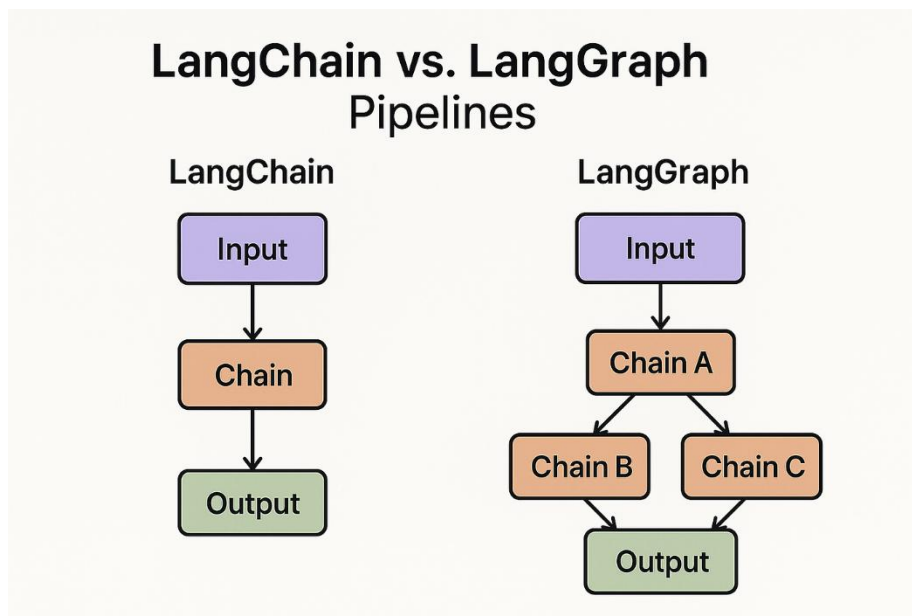


Рисунок 1.2 – Сравнение LangChain и LangGraph систем

Основная идея заключается в том, чтобы сделать поведение агентов детерминированным и отлаживаемым даже в условиях неопределённости и длительных последовательностей действий. Состояние графа (state) хранится централизованно и может быть персистентным – сохранённым между запусками, что критично для задач, требующих многочасовой или многодневной обработки.

1.3.2 Граф в LangGraph строится из узлов и рёбер. Узел может быть функцией, цепочкой, агентом или специальным блоком. Рёбра определяют не только последовательность, но и условия перехода: условные рёбра позволяют динамически выбирать следующий узел на основе текущего состояния или результата предыдущего шага. Состояние графа передаётся между узлами в виде объекта, который может содержать любые данные: от простых строк до сложных структур с историей сообщений, промежуточными результатами и метаданными.

Важные механизмами являются:

- сохранение промежуточных состояний (checkpointing);
- приостановка выполнения для получения ввода человека (human-in-the-loop);
- потоковая передача промежуточных результатов(streaming);
- асинхронное выполнение и встроенную обработку ошибок через retry-политики и fallback-узлы.

LangGraph поддерживает интеграцию с LangSmith для полной трассировки каждого пути выполнения графа, визуализации зависимостей и

сравнения различных вариантов workflow. Это позволяет проводить А/В-тестирование стратегий агентов и находить узкие места в реальном времени.

1.3.3 В системах интеллектуального подбора персонала LangGraph используется для формализации полного цикла обработки кандидата как управляемого графа. Типичный workflow может включать следующие узлы: сбор данных (парсинг резюме и профилей из внешних источников), извлечение и нормализация сущностей (опыт, навыки, образование), генерация векторных представлений, гибридный поиск по базе кандидатов, ранжирование с учётом дополнительных фильтров, генерация объяснения релевантности и, при необходимости, запрос уточнения у рекрутера.

1.4 Архитектурный паттерн Retrieval-Augmented Generation

1.4.1 Архитектурный паттерн Retrieval-Augmented Generation (RAG) представляет собой гибридный подход к построению систем искусственного интеллекта, который комбинирует возможности больших языковых моделей с внешними базами знаний [10]. Основная проблема традиционных языковых моделей заключается в ограниченности их знаний временным срезом данных на момент обучения и склонности к генерации фактологических ошибок, так называемых галлюцинаций. Паттерн RAG решает эту задачу путем динамического извлечения релевантной информации из внешнего источника перед генерацией ответа. В контексте интеллектуального подбора персонала это позволяет системе оперировать актуальными данными о кандидатах, хранящимися в векторной базе, а не полагаться исключительно на внутренние знания модели. Схема архитектурного паттерна приведена на рисунке 1.3.

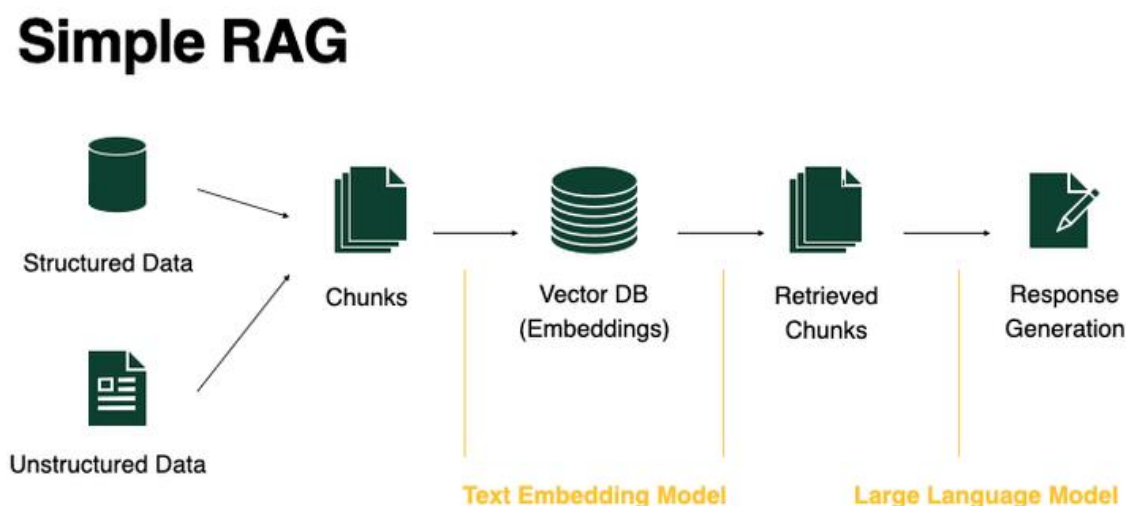


Рисунок 1.3 – Схема архитектурного паттерна RAG

Процесс работы систем с применением RAG архитектуры делится на три ключевых этапа. На этапе индексации документы разбиваются на смысловые

фрагменты, преобразуются в векторные представления и сохраняются в хранилище. На этапе поиска при поступлении запроса система формирует эмбединг запроса и находит наиболее близкие по семантике фрагменты в базе данных. На этапе генерации найденные фрагменты добавляются в контекст запроса к языковой модели, которая формирует итоговый ответ на основе предоставленных фактов.

1.4.2 Современное развитие паттерна RAG предполагает переход от наивного поиска к продвинутым стратегиям, таким как гибридный поиск и повторное ранжирование [11]. Гибридный поиск комбинирует семантическое сходство векторов с традиционным полнотекстовым поиском по ключевым словам, что позволяет учитывать как смысловую близость, так и точное совпадение терминов, например названий конкретных технологий или должностей. Механизм повторного ранжирования применяет более тяжелые и точные модели к отобранному короткому списку кандидатов, чтобы уточнить порядок выдачи результатов. Особое значение для многоагентных систем имеет концепция Agentic RAG, где агент самостоятельно планирует стратегию поиска, выбирает необходимые инструменты и источники данных, а также критически оценивает полученную информацию перед формированием ответа. Такой подход обеспечивает гибкость системы и позволяет обрабатывать сложные многошаговые запросы рекрутеров, требующие анализа различных аспектов профиля кандидата.

1.4.3 Качество работы системы Retrieval-Augmented Generation напрямую зависит от качества исходных данных, что делает задачи парсинга и обработки документов критически важными. Резюме и сопроводительные письма часто поступают в форматах, сложных для автоматической обработки, таких как PDF с многоколоночной версткой, таблицами, вложенными элементами и смешанным контентом. Традиционные методы извлечения текста, основанные на простых парсерах, часто теряют структурную информацию, нарушают логический порядок блоков или некорректно интерпретируют специальные символы. Для решения этих задач в разрабатываемой системе применяются современные инструменты документного интеллекта, обеспечивающие сохранение семантической структуры исходных материалов.

1.5 Модели для семантического анализа и формирования векторных представлений

1.5.1 Семантический анализ текстов в современных системах искусственного интеллекта опирается на преобразование естественного языка в числовые векторы фиксированной размерности – эмбединги (embeddings). Эти векторы размещаются в многомерном пространстве таким образом, что фрагменты текста, близкие по смыслу, оказываются геометрически близкими друг к другу. Мерой близости чаще всего выступает косинусное сходство или

евклидово расстояние, которое позволяет количественно оценивать семантическую релевантность даже при отсутствии общих лексических единиц, учитывая синонимию, контекстные оттенки и структурные особенности высказывания.

Эмбединги решают фундаментальную проблему несоответствия между дискретной природой слов и непрерывной природой смысла, позволяя машинам оперировать текстом на уровне абстрактных понятий. Эмбединги снижают размерность исходных данных: из тысяч или миллионов токенов формируется компактный вектор (обычно от 384 до 4096 измерений), который сохраняет основную семантическую информацию и может быть эффективно индексирован и сравнен.

1.5.2 Исторически эмбединги прошли три основных этапа развития. Первое поколение – частотные модели (TF-IDF, Bag-of-Words) – опиралось исключительно на статистику встречаемости слов в документе и корпусе, полностью игнорируя семантику и порядок. Второе поколение – предсказательные модели (Word2Vec с архитектурами CBOW и Skip-gram, GloVe, FastText) – использовало нейронные сети для обучения статических векторов на основе локального контекста или глобальной коокуренции слов [12]. Эти модели впервые позволили захватывать семантические отношения, но оставались статичными: каждое слово получало один и тот же вектор независимо от окружающего контекста.

Третье поколение – контекстуальные модели на базе трансформеров – радикально изменило ситуацию. BERT ввёл двунаправленное внимание и предобучение по маскированным токенам, позволяя генерировать разные представления одного и того же слова в зависимости от предложения [13]. Последующие модели (RoBERTa, ELECTRA, T5, DeBERTa) улучшали качество за счёт большего объёма данных, более сложных задач предобучения и оптимизаций. Современные модели, оптимизированные специально для задач поиска и retrieval (E5, BGE, GTE, Snowflake Arctic Embed), используют контрастивное обучение на парах (positive и negative примеры), чтобы максимизировать разделимость релевантных и нерелевантных документов в векторном пространстве.

1.5.3 Для задач, требующих обработки текстов на нескольких языках применяются мультязычные модели эмбедингов. Они обучаются на параллельных или смешанных многоязычных корпусах и формируют единое векторное пространство, в котором документы на разных языках могут сравниваться напрямую без предварительного перевода.

Семейство Qwen3-Embedding, разработанное на базе фундаментальных моделей Qwen3/Qwen3.5, занимает лидирующие позиции среди открытых мультязычных решений. Модели серии (0,6 млрд, 4 млрд и 8 млрд) демонстрируют высокие результаты на многоязычном рейтинге MTEB, особенно в задачах поиска по смыслу (retrieval), оценки семантической близости текстов (semantic textual similarity) и классификации

[14]. Qwen3-Embedding поддерживает более 100 языков, включая русский, обладает длиной контекста до 32 тысяч токенов, умеет учитывать инструкции (instruction-aware) и применяет технологию Matryoshka Representation Learning, которая обеспечивает гибкую размерность выходных векторов. Эти характеристики делают данное семейство моделей оптимальным выбором для проекта, в котором необходимо точно сопоставлять резюме и описания вакансий на разных языках.

1.6 Векторные базы данных для хранения и поиска

1.6.1 Векторные базы данных предназначены для хранения и быстрого поиска в массивах высокоразмерных векторов, представляющих семантические представления текстов, изображений или других объектов. В отличие от традиционных реляционных или документных баз, где поиск выполняется по точному совпадению или по ключевым словам, векторные базы ориентированы на приближённый поиск ближайших соседей (approximate nearest neighbors, ANN). Это позволяет находить наиболее семантически близкие записи даже при отсутствии буквального совпадения.

Основной алгоритмический приём – использование индексов приближённого поиска, среди которых наиболее распространён HNSW (Hierarchical Navigable Small World) [15], схема приведена на рисунке 1.4. Этот метод строит многоуровневый граф, где каждый узел (вектор) связан с ближайшими соседями на разных уровнях иерархии. Поиск начинается с верхнего уровня и постепенно уточняется на нижних, что обеспечивает баланс между скоростью и точностью.

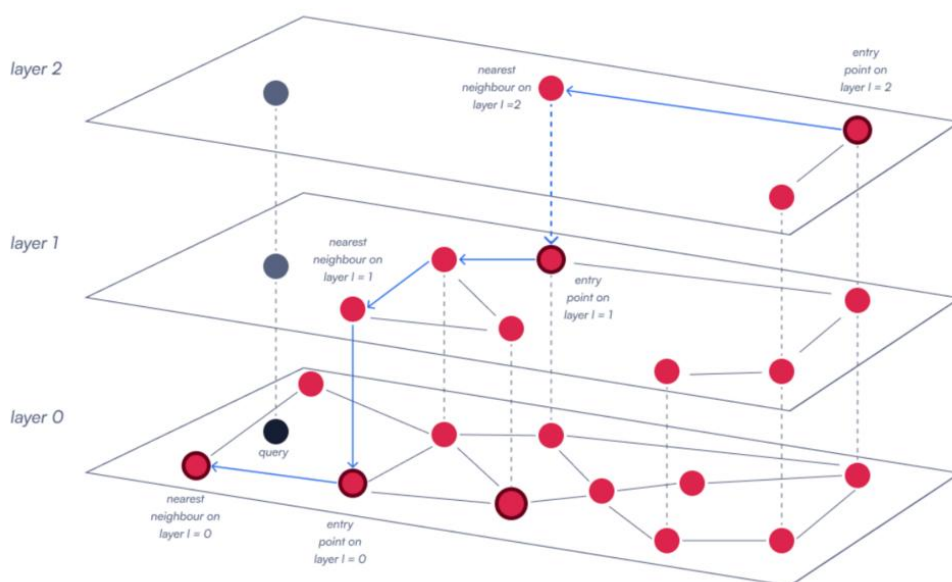


Рисунок 1.4 – HNSW алгоритм

Векторные базы также поддерживают хранение сопутствующих метаданных (payload), фильтрацию по атрибутам до или после поиска и

гибридный режим, когда векторный поиск сочетается с традиционным полнотекстовым (BM25, TF-IDF) или структурным. Это делает их незаменимыми в задачах, где семантика является лишь одним из критериев релевантности.

1.6.2 Qdrant – это высокопроизводительная открытая векторная база данных, написанная на языке Rust и ориентированная на низкую задержку и высокую пропускную способность [16]. Qdrant остаётся одним из лидеров среди открытых решений благодаря сочетанию скорости, надёжности и удобства интеграции с экосистемой Python и LangChain.

База использует собственную реализацию HNSW с оптимизациями для кэширования, сжатия векторов и параллельного выполнения запросов. Qdrant поддерживает хранение произвольных метаданных в формате `payload`, что позволяет прикреплять к каждому вектору профиль кандидата: опыт работы, навыки, желаемая зарплата, регион, уровень владения языками. Фильтрация по этим полям может выполняться как до поиска (*pre-filtering*), так и после (*post-filtering*), что критично для HR-задач, где нужно соблюдать жёсткие ограничения. Графический интерфейс с отображением векторов и полезной нагрузки отображен на рисунке 1.5.

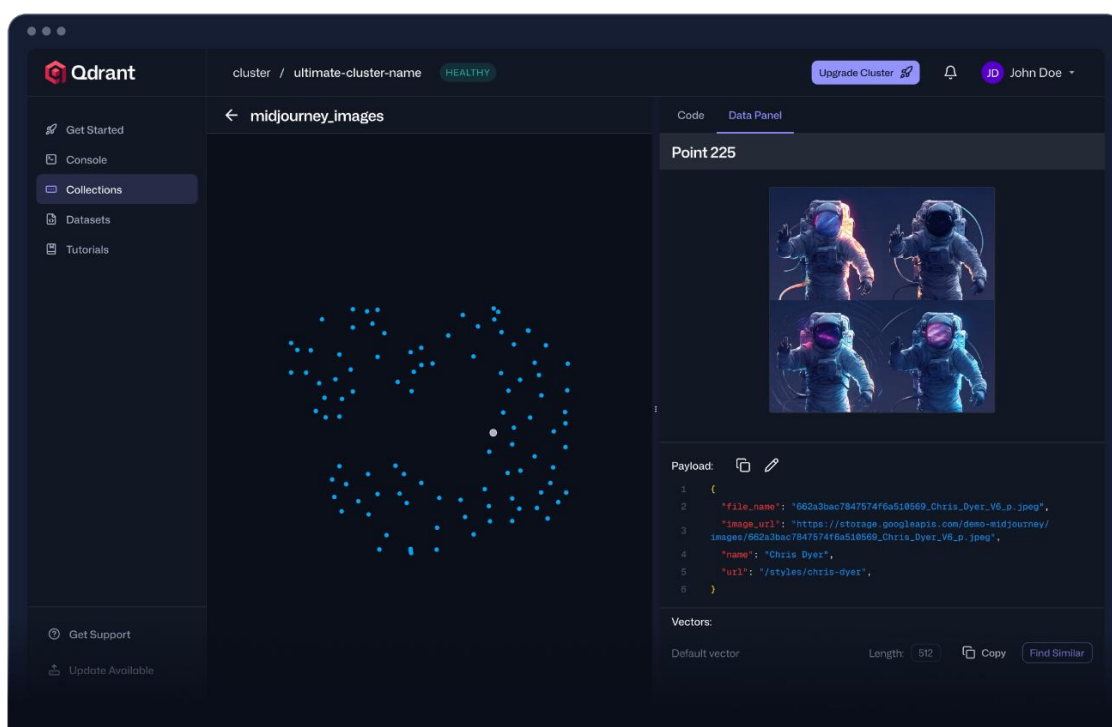


Рисунок 1.5 – Графический интерфейс Qdrant

Гибридный поиск в Qdrant объединяет векторную близость с полнотекстовым индексом, позволяя ранжировать результаты по комбинированной метрике. Дополнительные возможности включают горизонтальное масштабирование за счёт кластеризации, репликацию данных, создание моментальных снимков состояния базы в определённый момент

времени (point-in-time snapshots) и встроенную поддержку интерфейсов REST и gRPC. Интеграция с платформой LangChain выполняется через официальный адаптер хранилища векторов (vectorstore-адаптер), что существенно упрощает применение базы в многоагентных рабочих процессах (workflow).

1.6.3 В проекте интеллектуальной системы подбора персонала Qdrant выполняет роль центрального хранилища профилей кандидатов. После парсинга резюме и формирования векторных представлений с помощью модели Qwen3-Embedding каждый профиль индексируется как точка (point) с соответствующим payload. При поступлении запроса (описание вакансии) система генерирует вектор запроса и выполняет гибридный поиск: семантическая близость определяет основных кандидатов, а фильтры по атрибутам отсекают неподходящих.

Возможность обновления и удаления точек в реальном времени обеспечивает актуальность базы при поступлении новых резюме или изменении статуса кандидата.

1.7 Обзор аналогов

Системы подбора персонала с применением искусственного интеллекта представляют собой комплексные платформы, которые интегрируют семантический анализ, предиктивную аналитику и автоматизацию взаимодействия с кандидатами. Они ориентированы на крупные и средние организации, где требуется обработка больших объёмов заявок и поддержка полного цикла найма.

1.7.1 Resume Matcher – активно развивающийся открытый проект, предназначенных для сравнения резюме с описанием вакансии [17]. Инструмент выполняет двухуровневый анализ: сначала по ключевым словам и совпадению терминов, затем по семантической близости с использованием современных моделей векторных. Результаты визуализируются в виде тепловой карты совпадений, процентного рейтинга релевантности и списка недостающих или избыточных компетенций. Проект также предлагает рекомендации по доработке резюме для повышения вероятности прохождения автоматических фильтров ATS.

Решение ориентировано как на соискателей (помогает оптимизировать резюме под конкретную вакансию), так и на небольшие команды рекрутеров, которым требуется быстрый и бесплатный инструмент локального анализа. Преимущества включают простоту установки (часто через Docker или pip), полную прозрачность кода, отсутствие зависимости от облачных сервисов и возможность доработки под конкретные языки или домены. Ограничения – отсутствие встроенной базы кандидатов, многоагентной логики и масштабируемости для больших объёмов данных. Интерфейс продукта приведен на рисунке 1.6.

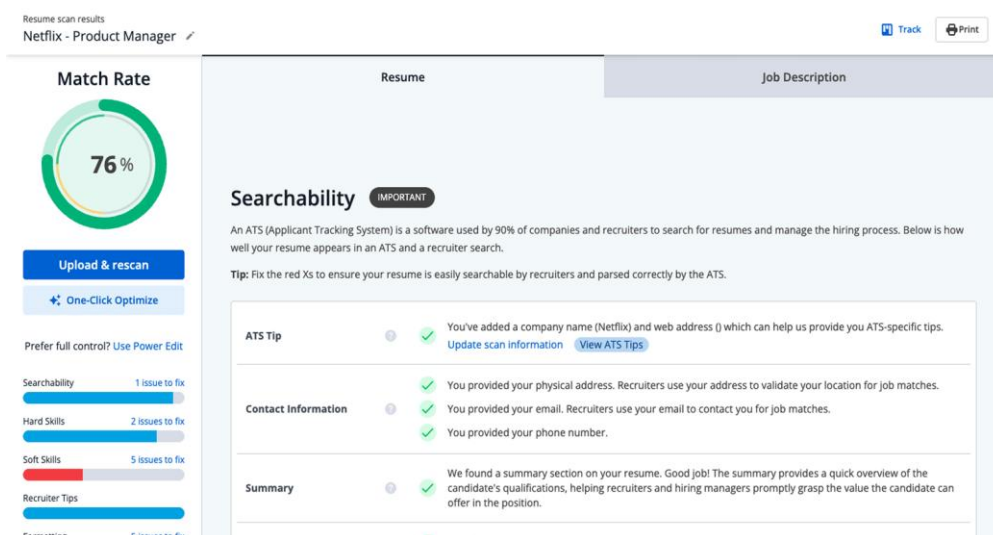


Рисунок 1.6 – Интерфейс Resume Matcher

1.7.2 Eightfold AI – одна из ведущих платформ консалтинговой компании Talent Intelligence, которая строит глубокие профили кандидатов на основе их навыков, профессионального опыта, образования и карьерных траекторий [18]. Система использует графовые представления знаний, где каждый кандидат связан с тысячами навыков, проектов и ролей. Это позволяет выполнять точное семантическое сопоставление с вакансиями, прогнозировать вероятность успеха на позиции, а также оценивать потенциал внутренней мобильности сотрудников. Интерфейс продукта представлен на рисунке 1.7.

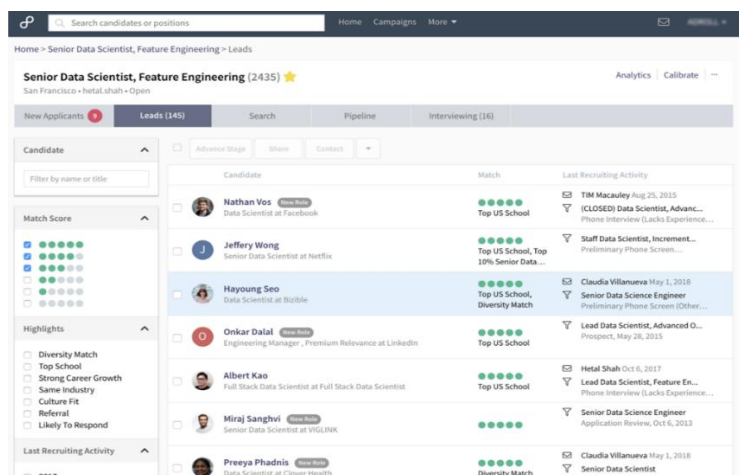


Рисунок 1.7 – Интерфейс Eightfold AI

Платформа включает механизмы автоматического снижения предвзятости, интеграцию с большинством популярных ATS и HRIS-систем, а также инструменты аналитики подбора. Eightfold особенно эффективна в крупных корпорациях, где требуется единый взгляд на весь пул талантов компании. Основные преимущества – высокая точность матчинга и поддержка

как внешнего набора, так и развития внутренних сотрудников. Ограничения связаны с высокой стоимостью внедрения и лицензирования, а также с частичной закрытостью алгоритмов. Интерфейс продукта приведен на рисунке 1.7.

1.7.3 Phenom – комплексная платформа Talent Experience Management, охватывающая весь жизненный цикл взаимодействия с кандидатами и сотрудниками [19]. Система включает создание персонализированных карьерных сайтов, чат-ботов для первичного общения, автоматизированный скрининг резюме, интеллектуальное планирование собеседований и аналитику вовлечённости на всех этапах. Семантический матчинг сочетается с предиктивными моделями, которые оценивают не только текущее соответствие, но и долгосрочный потенциал кандидата. Интерфейс продукта представлен на рисунке 1.8.

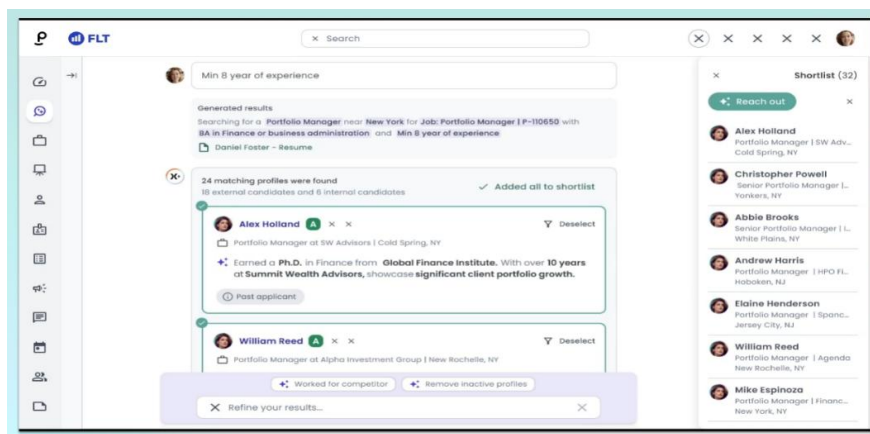


Рисунок 1.8 – Интерфейс Phenom

Phenom особенно сильна в сценариях высоконагруженного найма (розничная торговля, логистика, колл-центры, гостиничный бизнес), где ежедневно поступают тысячи заявок. Платформа активно использует персонализацию коммуникаций и автоматизацию рутинных задач рекрутеров. Преимущества – удобный пользовательский интерфейс, встроенная аналитика и поддержка полного цикла talent journey. Недостатки – высокая стоимость и необходимость значительной настройки под корпоративные процессы.

1.7.4 Коммерческие платформы обеспечивают готовое решение с высокой степенью автоматизации, поддержкой и соответствием корпоративным стандартам, однако требуют значительных финансовых вложений и ограничивают доступ к внутренней логике алгоритмов. Open-source инструменты, такие как Resume Matcher, предлагают гибкость и отсутствие затрат, но не покрывают полный цикл обработки и требуют самостоятельной доработки.

Разрабатываемая система занимает промежуточное положение: она использует открытые компоненты (LangChain, LangGraph, Qdrant, Qwen3-

Embedding), обеспечивая высокую кастомизацию, мультиязычность и прозрачность всех этапов. При этом проект реализует полноценный многоагентный рабочий процесс с семантическим анализом, гибридным поиском и возможностью человеческого контроля, что позволяет достичь уровня функциональности, сопоставимого с коммерческими решениями, при существенно меньших затратах и полной независимости от внешних поставщиков.

1.8 Технологии парсинга и обработки документов

1.8.1 Эффективность работы интеллектуальной системы подбора персонала напрямую зависит от качества извлечения информации из исходных документов. Резюме и сопроводительные письма поступают в систему в разнородных форматах. Каждый формат имеет свои особенности хранения структуры и метаданных. Файлы PDF могут быть как текстовыми, так и графическими, содержать многоколоночную верстку, таблицы и встроенные изображения. Документы DOCX хранят структуру в виде XML-тегов, что облегчает доступ к стилям и разделам, но требует специфических парсеров. Задача обработки заключается не только в извлечении текстового содержимого, но и в сохранении логической структуры документа, такой как заголовки разделов, списки навыков, хронология опыта работы и контактная информация. Потеря структурной информации на этапе парсинга приводит к ошибкам в последующей векторизации и семантическом поиске, поэтому выбор инструментария является критическим этапом проектирования.

1.8.2 Среди традиционных программных библиотек для работы с документами широко распространены инструменты с детерминированной логикой извлечения. Для формата PDF популярным решением является библиотека `pdfplumber` [20], которая строит геометрическую модель страницы на основе координат текстовых блоков. Этот подход позволяет точно определять границы колонок и таблиц, что критично для резюме со сложной версткой. Преимуществом таких решений является высокая скорость работы и низкие вычислительные затраты, так как не требуется запуск тяжелых нейронных сетей. Для формата DOCX стандартным инструментом выступает `python-docx`, обеспечивающий доступ к параграфам, стилям и таблицам через объектную модель документа. Однако детерминированные методы имеют существенное ограничение: они плохо справляются с неструктурированными данными, сканированными копиями документов или файлами с нарушенной кодировкой. В таких случаях извлеченный текст может быть фрагментирован или лишен смысловых связей между элементами.

1.8.3 В последнее время набирают популярность решения класса Document AI, использующие модели машинного обучения для понимания макета документа. Одним из современных представителей этого класса является библиотека `Docling` [21], разработанная для анализа сложных

документов с сохранением иерархии элементов, схема обработки представлена на рисунке 1.9. Инструмент применяет комбинацию компьютерного зрения и языкового моделирования для распознавания заголовков, абзацев, списков и таблиц, преобразуя их в структурированный формат, удобный для дальнейшей обработки. Аналогичный подход реализован в модели LayoutLM, которая обучается на совместном представлении текста и визуальных признаков страницы [22]. Такие технологии демонстрируют высокую устойчивость к вариативности шаблонов резюме и способны корректно интерпретировать данные даже при отсутствии явной разметки. Недостатком данных решений является повышенная требовательность к вычислительным ресурсам и большее время обработки одного документа по сравнению с традиционными парсерами.

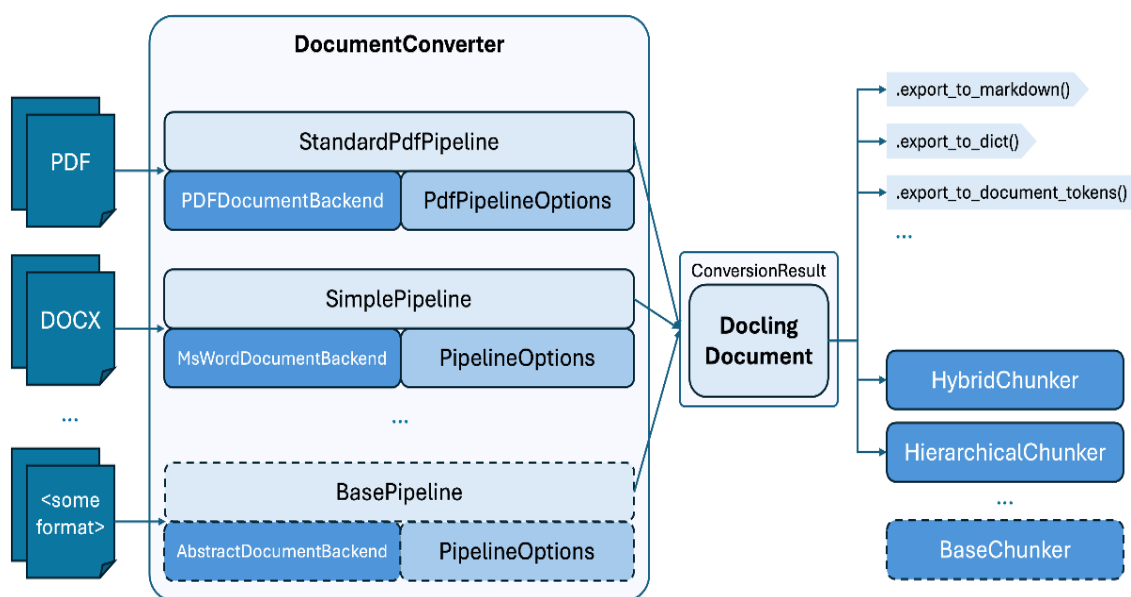


Рисунок 1.9 – Схема Docling

1.8.4 Существует также класс универсальных конвейеров обработки, таких как фреймворк Unstructured, который объединяет множество стратегий парсинга в единый интерфейс [23]. Подобные системы автоматически определяют тип файла и выбирают оптимальный метод извлечения, переключаясь между детерминированными и нейросетевыми подходами в зависимости от сложности документа. Это позволяет обеспечить баланс между производительностью и точностью. Для задач интеллектуального подбора персонала, где важна максимальная полнота данных о кандидате, наиболее перспективным представляется гибридный подход. Он предполагает использование легких библиотек для стандартных случаев и подключение тяжелых моделей анализа макета для сложных документов. Такой подход позволяет минимизировать потери информации на этапе первичной обработки и обеспечить высококачественную подготовку данных для генерации векторных представлений.

2 СИСТЕМНОЕ ПРОЕКТИРОВАНИЕ

В разработке интеллектуальной системы подбора персонала ключевым этапом является проектирование архитектуры, которая должна обеспечивать автоматическую обработку неструктурированных данных, глубокое семантическое понимание компетенций кандидатов и высокоточную выдачу наиболее релевантных соискателей. Процесс системного проектирования включает определение основных функциональных блоков, чёткое распределение ответственности между ними, установление взаимосвязей и способов взаимодействия компонентов.

После всестороннего анализа требований и задач дипломного проекта было принято решение разделить систему на следующие логические блоки:

- блок взаимодействия с пользователем;
- блок обеспечения безопасности и контроля доступа;
- блок агрегации и первичной обработки данных;
- блок парсинга и извлечения сущностей;
- блок генерации векторных представлений;
- блок векторного хранилища и индексации;
- блок гибридного поиска и ранжирования;
- блок оркестрации и координации агентов.

Структурная схема с перечисленными блоками представлена на чертеже ГУИР.400201.016 С1. Каждый блок подробно рассматривается в отдельном пункте.

2.1 Блок взаимодействия с пользователем

Блок взаимодействия с пользователем представляет собой интерфейсный слой системы, через который осуществляется всё общение кадрового специалиста с интеллектуальной системой подбора персонала. Его главная задача в обеспечении максимально удобного и интуитивно понятного процесса работы с системой на всех этапах: от формирования запроса до получения и анализа результатов.

Основные функции блока включают приём текстового запроса в свободной формулировке на естественном языке, применение дополнительных фильтров по опыту работы, уровню заработной платы, локации, обязательным и желательным компетенциям, загрузку новых резюме как в единичном, так и в пакетном режиме, а также отображение ранжированного списка кандидатов с визуализацией степени их соответствия вакансии и пояснениями причин релевантности. Кроме того, блок предоставляет возможность экспорта результатов в удобных форматах для дальнейшего использования.

Блок взаимодействия с пользователем передаёт сформированный запрос и загруженные документы в блок оркестрации и координации агентов, а затем получает от него полностью обработанные и структурированные результаты для их последующей визуализации и представления пользователю.

2.2 Блок обеспечения безопасности и контроля доступа

Блок обеспечения безопасности и контроля доступа реализует комплекс механизмов, направленных на защиту системы и всех обрабатываемых в ней данных. Он отвечает за аутентификацию, авторизацию и аудит действий, обеспечивая надёжную защиту от несанкционированного доступа на всех уровнях системы.

В основе работы блока лежит модель сервисных токенов, которые используются для подтверждения полномочий при взаимодействии между компонентами. Полученные токены передаются во все остальные блоки и позволяют осуществлять строгий контроль доступа к операциям в зависимости от роли и идентификатора вызывающего сервиса, а также ведут детальный аудит ключевых событий.

Благодаря данному блоку в системе формируется единый, целостный контур безопасности, который охватывает все компоненты и процессы. Это позволяет гарантировать конфиденциальность обрабатываемых персональных данных и предотвращать любые попытки несанкционированного использования функциональности системы.

2.3 Блок агрегации и первичной обработки данных

Блок агрегации и первичной обработки данных выступает в роли входного шлюза системы, через который поступают все документы для последующего анализа. Его задача заключается в обеспечении унификации и предварительной подготовки разнородных входных данных перед их глубоким структурным разбором.

Ключевые функции блока включают приём файлов резюме в различных форматах, определение типа документа и языка, нормализацию кодировок, очистку от очевидных артефактов, первичную категоризацию материалов и их маршрутизацию в следующий этап обработки. После завершения первичной обработки подготовленные данные направляются в блок парсинга и извлечения сущностей.

Правильная работа данного блока является необходимым условием для эффективной и стабильной работы всей системы, поскольку именно здесь закладывается основа качества дальнейшего семантического анализа.

2.4 Блок парсинга и извлечения сущностей

Блок парсинга и извлечения сущностей выполняет глубокий анализ неструктурированного текстового содержимого документов с целью создания формализованного профиля кандидата. Этот блок является одним из ключевых элементов системы, отвечающих за переход от необработанных данных к структурированной информации.

Основные задачи блока заключаются в распознавании структуры документа, извлечении именованных сущностей и атрибутов, нормализации

компетенций к единому контролируемому словарю, формировании строго типизированной модели профиля кандидата, а также обнаружении и обработке противоречий и неполноты данных. Полученный структурированный профиль передаётся в блок генерации векторных представлений и в блок оркестрации для дальнейшей обработки.

2.5 Блок генерации векторных представлений

Блок генерации векторных представлений отвечает за преобразование текстового и структурированного содержимого профиля кандидата в числовые векторные эмбединги, которые сохраняют семантическую близость информации. Данный блок обеспечивает возможность последующего сравнения кандидатов не только по формальным признакам, но и по смысловому соответствию.

Основные функции включают генерацию векторных представлений как всего профиля в целом, так и отдельных его значимых секций. Полученные эмбединги вместе с сопутствующими метаданными передаются в блок векторного хранилища и индексации, а при необходимости возвращаются в блок оркестрации и координации агентов.

2.6 Блок векторного хранилища и индексации

Блок векторного хранилища и индексации предназначен для долговременного хранения векторных представлений профилей кандидатов совместно с ассоциированными метаданными. Он обеспечивает эффективный и быстрый доступ к накопленным данным для выполнения поисковых операций.

Ключевые функции блока включают хранение векторов и связанных с ними данных, поддержку быстрого приближённого поиска по сходству, индексацию и фильтрацию по структурированным метаданным, а также инкрементальное обновление и версионирование профилей. Блок получает данные от блока генерации векторных представлений и предоставляет доступ к индексу для блока гибридного поиска и ранжирования.

2.7 Блок гибридного поиска и ранжирования

Блок гибридного поиска и ранжирования выполняет основную поисковую логику системы, объединяя несколько типов поиска и многоступенчатое ранжирование для достижения максимальной релевантности результатов. Именно этот блок отвечает за итоговое качество подбора кандидатов.

Основные механизмы включают первичный отбор кандидатов, применение дополнительных моделей переранжирования при необходимости, скоринг с учётом бизнес-правил и формирование финального упорядоченного списка с оценками степени соответствия. Блок получает запрос от блока

оркестрации, взаимодействует с блоком векторного хранилища и возвращает результаты через блок оркестрации в блок взаимодействия с пользователем.

2.8 Блок оркестрации и координации агентов

Блок оркестрации и координации агентов является центральным управляющим компонентом всей системы. Он реализует динамическое планирование и исполнение многошаговых процессов обработки запросов, обеспечивая адаптивность системы к различным сценариям работы.

Основные функции блока включают анализ входного запроса и определение оптимального сценария обработки, диспетчеризацию задач специализированным блокам, условную маршрутизацию, управление состоянием процесса, обработку исключительных ситуаций, агрегацию промежуточных результатов и формирование финального ответа. Блок получает начальный запрос от блока взаимодействия с пользователем, координирует работу всех остальных компонентов системы и возвращает полностью обработанный результат обратно в интерфейсный слой.

Таким образом, блок оркестрации обеспечивает целостность и логическую связность всей архитектуры, реализуя многоагентный подход к решению сложных задач подбора персонала.

2.9 Общие принципы взаимодействия блоков

Взаимодействие блоков системы построено на принципах модульности и централизованной координации. Каждый компонент отвечает за строго ограниченную задачу, а все ключевые процессы управляются через блок оркестрации и координации агентов. Он анализирует запрос, распределяет задачи по специализированным блокам, собирает промежуточные результаты и формирует финальный ответ, минимизируя прямые связи между остальными компонентами и упрощая трассировку и управление.

Обработка начинается с интерфейсного блока, откуда запрос поступает в оркестратор. Далее следует первичная очистка и унификация данных, парсинг и структурирование, генерация векторных представлений, сохранение в индексированном хранилище. При поиске оркестратор передаёт задачу блоку гибридного поиска и ранжирования, который обращается к хранилищу, комбинирует разные подходы и возвращает ранжированный список кандидатов с пояснениями.

Такая структура обеспечивает гибкость, предсказуемость и масштабируемость: логика процессов легко меняется в оркестраторе, блоки остаются независимыми. В результате система эффективно справляется с разнообразными задачами при высокой вариативности данных и требований.

3 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ

В данном разделе описываются структура разрабатываемого программного обеспечения с точки зрения данных и обрабатывающих их подпрограмм. В частности, рассматриваются модели данных, соответствующие им сущности и их взаимодействие, а также структура проекта.

Помимо функциональной структуры, важной частью разработки программного обеспечения является архитектурная структура. Архитектурная структура определяет общую организацию программы, ее компоненты и способы взаимодействия между ними. Она описывает, как различные части программного обеспечения взаимодействуют друг с другом и как они организованы для достижения целей приложения.

Система реализована в виде пяти микросервисов на языке Python с использованием фреймворка FastAPI и одного frontend-сервиса на NestJS (с React-приложением и архитектурой микрофронтонтов Module Federation). Каждый сервис отвечает за строго определённую часть функционала и построен по принципам микросервисной архитектуры.

Функциональная структура состоит из следующих микросервисов:

- `extraction_service` – блок агрегации, первичной обработки, парсинга и генерации векторных представлений;
- `storage_service` – блок векторного хранилища и индексации;
- `search_service` – блок гибридного поиска и ранжирования;
- `orchestrator_service` – блок оркестрации и координации агентов (центральный мозг системы);
- `frontend` – блок взаимодействия с пользователем;
- `auth_service` – блок обеспечения безопасности и контроля доступа.

3.1 Описание сервиса `extraction_service`

Данный сервис объединяет этапы загрузки документов, извлечения текста, обогащения метаданными и генерации эмбеддингов. Он является входным шлюзом системы и обеспечивает переход от неструктурированных файлов к векторным представлениям.

3.1.1 Класс `S3Service` представляет собой слой взаимодействия с объектным хранилищем и обеспечивает загрузку, хранение, удаление и получение файлов резюме. Он является входной точкой для всех документов, поступающих в систему, и отвечает за унификацию работы с S3-совместимым хранилищем.

Поле `_s3_client` содержит экземпляр клиента `aioboto3`, который используется для выполнения всех операций с бакетом.

Поле `_config` хранит настройки бакета, папки хранения и региона из конфигурационного сервиса.

Метод `async upload_file` принимает объект `UploadFile` и ключ файла, выполняет загрузку в бакет, логирует событие и возвращает ключ объекта для дальнейшей обработки.

Метод `async delete_file` удаляет объект по указанному ключу.

Метод `async get_file` возвращает поток файла, тип содержимого и размер для передачи пользователю или последующей конвертации.

Метод `async get_converted` конвертирует документ в формат PDF при помощи LibreOffice, если исходный формат отличается от PDF, и возвращает результат в виде base64-строки.

Метод `async get_documents_list` выполняет пагинированный поиск файлов с применением фильтра по имени и возвращает список объектов с метаданными.

3.1.2 Класс `TextProcessingService` реализует полный пайплайн первичной обработки и обогащения документов. Он отвечает за извлечение текста, чанкинг, генерацию метаданных и подготовку данных для векторизации.

Поле `_openai` предоставляет доступ к модели языка для генерации тегов и тем.

Поле `_file_to_text_axios` отвечает за взаимодействие с внешним сервисом извлечения текста.

Поле `_embedding_service` генерирует векторные представления.

Поле `_qdrant_service` сохраняет векторы в хранилище.

Поле `_splitter` содержит экземпляр `RecursiveCharacterTextSplitter` из `LangChain`.

Метод `async extract_text` принимает файл, при необходимости конвертирует его в PDF и отправляет на внешний сервис для извлечения сырого текста.

Метод `async extract_tags_and_topics` вызывает модель языка с подготовленным промптом и возвращает списки тегов и тем в формате JSON.

Метод `async process_document` выполняет полный цикл обработки: извлечение текста, разбиение на чанки, обогащение каждого чанка тегами и темами, генерацию эмбеддингов и сохранение векторов в Qdrant.

Метод `async migrate_document` выполняет аналогичный цикл для миграции документов между модулями с предварительным удалением старых записей по фильтру `fileKey`.

3.1.3 Класс `EmbeddingService` отвечает за генерацию векторных представлений текстовых данных. Он обеспечивает пакетную обработку и нормализацию эмбеддингов перед сохранением в векторное хранилище.

Поле `_embedding_axios` содержит клиент для взаимодействия с внешним сервисом эмбеддингов.

Поле `_config` хранит размер батча и URL сервиса.

Метод `async generate_embeddings` принимает список текстов, разбивает его на батчи согласно настройкам, отправляет запросы во внешний сервис, нормализует полученные векторы и возвращает результат в виде списка списков чисел с плавающей точкой.

3.1.4 Класс `DataSourceProvider` является центральным оркестратором всех внешних источников данных. Он регистрирует и управляет загрузчиками из различных систем, обеспечивая единый интерфейс для импорта резюме независимо от их происхождения.

Поле `_loaders` содержит словарь зарегистрированных загрузчиков (`S3Loader`, `ArchiveLoader`, `HHLoader`, `HRISLoader` и др.).

Поле `_text_processing_service` ссылается на сервис обработки для передачи каждого полученного документа.

Метод `register_loader` регистрирует новый загрузчик по типу источника.

Метод `async import_from_source` принимает тип источника и конфигурацию, подключается к нему, выгружает резюме пакетами и передаёт каждое в `TextProcessingService.process_document`.

Метод `async sync_archive` выполняет полный импорт старого архива из локальных папок или zip-файлов с сохранением оригинальной структуры.

Метод `async sync_hh_api` подключается к API HeadHunter, получает резюме по заданным вакансиям или поисковым запросам и запускает их обработку.

Метод `async sync_hris` обеспечивает синхронизацию с корпоративными HR-системами (1C, Workable и др.) через их API.

Метод `async sync_email` импортирует резюме из почтовых ящиков по протоколу IMAP.

Таким образом, `extraction_service` полностью реализует блок агрегации и первичной обработки данных, поддерживая работу с множеством разнородных источников и обеспечивая их интеграцию в единую векторную базу системы.

3.2 Описание сервиса `storage_service`

Данный сервис обеспечивает долговременное хранение векторных представлений профилей кандидатов вместе с ассоциированными метаданными. Он отвечает за эффективный доступ к накопленным данным, поддержку приближённого поиска по сходству, фильтрацию по структурированным атрибутам и инкрементальное обновление коллекций. Сервис построен вокруг векторной базы Qdrant и полностью соответствует блоку векторного хранилища и индексации дипломного проекта.

3.2.1 Класс `QdrantService` представляет собой основной интерфейс взаимодействия с векторным хранилищем Qdrant. Он отвечает за создание и

управление коллекциями, вставку, поиск, удаление и фильтрацию векторных точек. Класс реализует все необходимые операции для обеспечения высокоскоростного семантического поиска и поддерживает работу с несколькими коллекциями одновременно.

Поле `_qdrant_client` содержит экземпляр `QdrantClient` из официальной библиотеки `qdrant-client`, который используется для всех REST-вызовов к базе.

Поле `_config` хранит параметры коллекции, включая имя коллекции, размер вектора, лимит поиска и пороговое значение `score`, загруженные из конфигурационного сервиса.

Метод `async ensure_collection_exists` проверяет наличие коллекции по имени и при её отсутствии создаёт новую с заданным размером вектора и метрикой расстояния `Cosine`.

Метод `async upsert_documents` принимает список векторных точек, разбивает их на пакеты по 50 элементов и выполняет пакетную вставку с сохранением идентификатора, вектора и полного `payload`.

Метод `async search_documents` принимает эмбединг запроса и необязательные фильтры, формирует условия `must` по полям `fileKey`, `tags` и `topics`, выполняет поиск и применяет дополнительную пороговую фильтрацию относительно лучшего найденного `score`.

Метод `async delete_documents` удаляет точки по переданному списку идентификаторов.

Метод `async delete_by_filter` удаляет точки по произвольному фильтру `Qdrant`, что используется при миграции документов и очистке старых записей.

3.3 Описание сервиса `search_service`

Данный сервис реализует основную поисковую логику системы, объединяя векторный поиск, фильтрацию по метаданным, переранжирование и многоступенчатое ранжирование. Он отвечает за достижение максимальной релевантности результатов и полностью соответствует блоку гибридного поиска и ранжирования дипломного проекта.

3.3.1 Класс `QueryService` является центральным компонентом поисковой логики и реализует полный пайплайн обработки запроса пользователя. Он координирует анализ запроса, многоуровневый поиск, переранжирование, генерацию ответа и оценку релевантности.

Поле `_openai` содержит клиент `OpenAI` для генерации вопросов, анализа тегов и оценки релевантности.

Поле `_rerank_axios` отвечает за взаимодействие с внешним сервисом переранжирования.

Поле `_embedding_service` генерирует векторное представление запроса. Поле `_qdrant_service` выполняет векторный поиск с фильтрами.

Поле `_s3_service` предоставляет доступ к списку всех загруженных файлов для анализа.

Метод `async analyze_query_for_tags` принимает текст запроса и режим контекста, вызывает модель языка с подготовленным промптом, извлекает списки `file_keys`, `tags` и `topics` и выполняет сопоставление `file_keys` с реальными ключами из S3.

Метод `async rerank_documents` принимает запрос и список документов, отправляет их на внешний `reranker` и возвращает отсортированный результат по `rerank_score`.

Метод `async generate_question` анализирует историю диалога и либо извлекает последний вопрос пользователя, либо генерирует его заново с помощью модели языка.

Метод `async process_document_query` является основным методом сервиса: он генерирует финальный вопрос, выполняет анализ тегов, запускает многоуровневый поиск с последовательными `fallback`-стратегиями – с полными фильтрами, только по `file_keys`, без фильтров, формирует контекст из чанков, вызывает генерацию ответа с поддержкой потоковой передачи через `WebSocket`, оценивает релевантность ответа отдельным запросом к модели языка и возвращает полный результат вместе с источниками, оценкой `relevance_pct` и метриками `latency`.

Таким образом, класс `QueryService` объединяет все этапы гибридного поиска и обеспечивает высокую точность подбора кандидатов за счёт комбинации векторного поиска, бизнес-фильтров и дополнительного переранжирования.

3.4 Описание сервиса `orchestrator_service`

Данный сервис является центральным управляющим компонентом всей системы и реализует многоагентный процесс обработки запросов. Он отвечает за анализ входного запроса, динамическое планирование сценариев обработки, диспетчеризацию задач другим сервисам, агрегацию результатов и формирование финального ответа.

3.4.1 Класс `RagService` выступает главным оркестратором системы и реализует многоагентный процесс обработки запросов на базе `LangGraph`. Он координирует взаимодействие всех остальных микросервисов, планирует сценарии обработки и управляет полным жизненным циклом пользовательского запроса – от приёма сообщения до сохранения результата, оценки качества и обновления статистики.

Поле `_graph` содержит скомпилированный объект `CompiledGraph` (на основе `StateGraph` из `LangGraph`), который определяет узлы-агенты, рёбра и условные переходы.

`_text_processing_service` – отвечает за обработку новых документов (вызов `extraction_service`).

`_query_service` – выполняет поиск, reranking и генерацию ответа (вызов `search_service`).

`_qdrant_service` и `_s3_service` – используются для операций с векторным хранилищем и объектным хранилищем.

`_database_service` – обеспечивает персистентное хранение истории разговоров, оценок обратной связи и статистики использования.

Метод `async add_document` загружает файл в S3, иницирует граф с начальным состоянием `query_type="new_document"`, запускает обработку через `text_processing_service` и сохранение векторов в Qdrant.

Метод `async delete_document` удаляет файл из хранилища и все связанные векторы из Qdrant.

Метод `async post_user_message` сохраняет входящее сообщение в базу, создаёт начальное состояние графа, запускает `graph.invoke()` с типом "search" и передаёт WebSocket-сокет для потоковой передачи ответа.

Метод `async process_document_query` – основной метод обработки поискового запроса: извлекает историю разговора (с учётом режима контекста auto/manual), запускает граф LangGraph, получает результат, сохраняет в базу вместе с `relevance_pct` (от `evaluator_agent`), обновляет тему разговора через LLM при необходимости и возвращает ответ пользователю (включая источники, метрики latency и кандидатов).

Метод `async migrate_qdrant` запускает миграцию всех документов в новую коллекцию Qdrant.

Методы `create_client`, `get_client_list`, `create_conversation`, `get_statistics` и `save_message_feedback` делегируют работу с персистентными данными в `DatabaseService`.

3.4.2 Агенты внутри скомпилированного графа LangGraph (основные узлы):

`query_analyzer_agent` – анализирует входной запрос с помощью LLM, определяет тип задачи (загрузка документа, поиск по вакансии, уточнение запроса, миграция), извлекает ключевые параметры (теги, требуемые фильтры) и обновляет состояние графа.

`retrieval_agent` – выполняет многоуровневый гибридный поиск (векторный с фильтрами по метаданным), вызывает `_query_service` и агрегирует релевантные чанки/профили кандидатов.

`reranker_agent` – применяет переранжирование, улучшает точность топ-k кандидатов.

`response_generator_agent` – формирует финальный ответ (текстовое объяснение + ранжированный список кандидатов + степень соответствия + пояснения), поддерживает потоковую передачу через WebSocket.

`evaluator_agent` – оценивает качество сгенерированного ответа, при необходимости иницирует возврат к `retrieval_agent` для уточнения контекста.

3.4.3 Помимо обычных рёбер граф включает условные переходы (conditional edges), которые обеспечивают динамическую маршрутизацию в зависимости от состояния.

После завершения работы агента `query_analyzer_agent` происходит маршрутизация в зависимости от определённого типа запроса. Если тип запроса — `new_document`, то обработка выполняется прямым вызовом `text_processing`, после чего граф завершает выполнение (END). Если тип запроса — `search`, то последовательность продолжается через узлы в следующем порядке: `retrieval`, затем `reranker`, затем `generator`, затем `evaluator`, после чего граф завершает выполнение (END). При этом после `evaluator` возможен возврат в начало цепочки поиска.

После завершения работы агента `evaluator_agent` выполняется проверка значения `relevance_pct`. Если полученное значение `relevance_pct` оказывается ниже установленного порогового значения, то управление передаётся обратно агенту `retrieval_agent` для повторного поиска и уточнения результатов (рефлексия и итеративное улучшение). Если же `relevance_pct` находится на уровне порога или выше, то выполнение графа завершается (END).

3.4.4 При поступлении запроса создаётся объект состояния `AgentState` (`TypedDict`), содержащий следующие ключевые поля: `messages` (история сообщений), `query_type` (тип запроса), `context` (контекст диалога и метаданные), `retrieved_docs` (найденные документы/чанки), `reranked_docs` (переранжированные результаты), `final_result` (финальный сформированный ответ), `relevance_score` (оценка релевантности).

Далее выполняется вызов метода `await self._graph.ainvoke(initial_state, config={"configurable": {"thread_id": conversation_id}})` — это обеспечивает сохранение состояния графа и возможность продолжения разговора в рамках одной сессии (`conversation_id` используется как идентификатор потока).

Граф последовательно (или с условными ветвлениями) проходит по узлам-агентам, на каждом шаге обновляя общее состояние. Промежуточные и финальные результаты сохраняются в базе данных PostgreSQL через методы `DatabaseService` для обеспечения воспроизводимости, аудита и последующего анализа.

Потоковая передача ответа (streaming) фронтенду в реальном времени реализована через WebSocket: метод `response_generator_agent` реализован как асинхронный генератор (`async generator`), что позволяет отправлять фрагменты ответа по мере их генерации без ожидания полного завершения обработки.

Таким образом, использование `LangGraph` в классе `RagService` позволяет реализовать динамическое многоагентное планирование, условную маршрутизацию, итеративное улучшение результатов и механизмы самокоррекции (reflection loop).

Этот подход существенно повышает интеллектуальность и надёжность системы за счёт способности самостоятельно уточнять запрос, переоценивать результаты и корректировать стратегию поиска в зависимости от качества промежуточных данных.

3.4.5 Класс `DatabaseService` обеспечивает хранение и получение всех персистентных сущностей системы с помощью `SQLAlchemy` и `PostgreSQL`. Он отвечает за работу с клиентами, разговорами, сообщениями, оценками обратной связи и статистикой использования.

Метод `async create_client` создаёт нового клиента и проверяет уникальность имени.

Метод `async create_conversation` создаёт новый разговор для клиента.

Метод `async create_message` сохраняет сообщение в историю разговора.

Метод `async update_message_content` обновляет содержимое сообщения после генерации ответа.

Метод `async update_conversation_topic` автоматически генерирует и сохраняет тему разговора.

Метод `async get_conversation_story` возвращает полную историю сообщений конкретного разговора.

Метод `async save_message_feedback` сохраняет оценку пользователя (`useful / not_useful / incomplete`) и причину.

Метод `async get_statistics` формирует аналитический отчёт по количеству ответов, средней релевантности, `latency` и распределению оценок за выбранный период.

3.5 Описание сервиса frontend

Сервис `frontend` реализует блок взаимодействия с пользователем и выступает в роли `API Gateway` системы. Он построен на фреймворке `NestJS` и использует архитектуру микрофронтон (Module Federation) для фронтенд-приложения. Это позволяет независимо разрабатывать и деплоить отдельные модули интерфейса (список кандидатов, чат, статистика, загрузка резюме), сохраняя при этом единый хост-приложение. Сервис обеспечивает приём запросов от пользователя, загрузку файлов, отображение ранжированных результатов с визуализацией степени соответствия, экспорт данных, а также потоковую передачу ответов через `WebSocket`.

3.5.1 Класс `RagController` является основным `REST`-контроллером сервиса `frontend`. Он обрабатывает все `HTTP`-запросы, поступающие от пользовательского интерфейса, и делегирует их в `orchestrator_service`.

Поле `ragService` содержит инжектированный сервис оркестрации.

Поле `authGuard` применяется для проверки JWT-токена, полученного от Keycloak.

Метод `@Post('upload')` с использованием `FileInterceptor` принимает файл резюме, проверяет права доступа и передаёт его в `orchestrator_service.addDocument`.

Метод `@Post('message')` принимает новое сообщение пользователя вместе с контекстом разговора и запускает процесс генерации ответа.

Метод `@Post('chatResponse')` обеспечивает получение готового ответа без WebSocket.

Метод `@Post('documents/delete')` удаляет документ по `fileKey`.

Метод `@Post('documents/get')` возвращает файл в виде `StreamableFile` для скачивания.

Метод `@Post('documents/list')` возвращает пагинированный список загруженных резюме с фильтрами.

Метод `@Post('qdrantMigrate')` инициирует миграцию коллекции.

Методы `createClient`, `createConversation`, `getClientList`, `getClientData` и `getConversation` отвечают за управление клиентами и историей чатов.

Метод `@Post('message/feedback')` сохраняет оценку пользователя (`useful / not_useful / incomplete`) вместе с причиной.

3.5.2 Класс `RagGateway` реализует WebSocket-шлюз на базе `@nestjs/websockets` и обеспечивает потоковую передачу ответов в реальном времени. Он подключён к `namespace /rag` и использует `Socket.io`.

Метод `handleConnection` логирует подключение клиента и отправляет событие `'ready'`.

Метод `handleDisconnect` фиксирует отключение.

Метод `@SubscribeMessage('sendMessage')` принимает `payload` с `conversationId`, контекстом и новым сообщением, создаёт уникальный `traceID`, оборачивает логирование в `logNamespace` и вызывает `ragService.postUserMessage` с передачей сокета для стриминга.

3.5.3 Frontend-приложение использует `Webpack Module Federation`. Хост-приложение (`shell`) построено на `NestJS + React`, а удалённые микрофронты (`candidate-list`, `chat-module`, `statistics-module`, `upload-module`) разрабатываются и деплоятся независимо. Каждый микрофронт регистрируется в хосте через динамический импорт и получает данные через `REST/WebSocket` от `RagController` и `RagGateway`.

3.6 Описание сервиса `auth_service`

Сервис `auth_service` реализует блок обеспечения безопасности и контроля доступа. Он построен на базе Keycloak (OAuth2 + OpenID Connect) и обеспечивает единую точку аутентификации для всех пользователей и

микросервисов системы. Сервис отвечает за аутентификацию кадровых специалистов, выдачу сервисных токенов, авторизацию межсервисных вызовов по ролям и детальный аудит всех операций.

3.6.1 Класс `KeycloakService` является основным провайдером аутентификации и взаимодействия с Keycloak-сервером.

Поле `keycloakAdminClient` содержит администраторский клиент Keycloak для управления пользователями и ролями.

Поле `jwtService` используется для валидации и генерации JWT.

Поле `config` хранит URL Keycloak, `realm`, `clientId` и `clientSecret`.

Метод `async authenticateUser` принимает `login` и `password`, выполняет прямой `grant password flow` и возвращает `access_token` и `refresh_token`.

Метод `async issueServiceToken` генерирует токен для межсервисного взаимодействия (`client_credentials grant`) с указанием `service-role`.

Метод `async validateToken` декодирует и проверяет JWT-токен, извлекает роли и `tenant`-информацию.

Метод `async getUserInfo` возвращает информацию о пользователе по `subject` из токена.

Метод `async logout` выполняет `logout` пользователя в Keycloak.

3.6.2 Класс `AuthGuard` реализует NestJS Guard и применяется ко всем защищённым эндпоинтам. Он извлекает токен из заголовка `Authorization`, валидирует его через `KeycloakService` и проверяет наличие необходимых ролей (например, `'hr-specialist'`, `'admin'`).

3.6.3 Класс `TokenService` отвечает за работу с сервисными токенами. Он генерирует короткоживущие `m2m`-токены для вызовов между `extraction_service`, `storage_service`, `search_service` и `orchestrator_service`, а также поддерживает их кэширование в Redis.

3.6.4 Класс `AuditLogger` фиксирует все ключевые события безопасности (успешная аутентификация, попытки доступа, смена ролей, удаление документов и т.д.). Каждое событие сохраняется в PostgreSQL с указанием `user_id`, `service_name`, `action`, `resource` и `timestamp`.

Таким образом, функциональная структура системы полностью соответствует требованиям дипломного проекта. Каждый сервис и класс имеет чётко определённую ответственность, что обеспечивает модульность, масштабируемость и высокую точность семантического подбора персонала.

4 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ

Текст

4.1 Текст

Текст

5 ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ

Текст раздела

5.1 Подраздел

Текст раздела

6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Текст раздела

6.1 Подраздел

Текст раздела

7 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ СИСТЕМЫ ПОДБОРА ПЕРСОНАЛА С СЕМАНТИЧЕСКИМ АНАЛИЗОМ АГРЕГИРОВАННЫХ ДАННЫХ

7.1 Характеристика программного средства, разрабатываемого для реализации на рынке

Разрабатываемое программное средство представляет собой интеллектуальную систему подбора персонала с семантическим анализом агрегированных данных, предназначенную для автоматизации процессов поиска и оценки кандидатов в организации. Основная задача разработки – обеспечить точный и быстрый подбор релевантных соискателей на основе глубокого понимания компетенций, опыта и требований вакансий, сократить время скрининга резюме и снизить субъективные ошибки, характерные для традиционных методов подбора на основе ключевых слов. Потребность в таком решении обусловлена экспоненциальным ростом объёмов неструктурированных данных о кандидатах, высокой конкуренцией за квалифицированные кадры и необходимостью повысить эффективность HR-процессов в условиях цифровой трансформации рынка труда.

Разработка выполняется в рамках дипломного проекта и ориентирована на реализацию в виде самостоятельного продукта для открытого рынка. Система создаётся на базе открытых технологий и рассчитана на многократное использование разными компаниями и специалистами по подбору персонала без привязки к процессам одной конкретной организации.

Ожидаемым результатом внедрения для пользователей является значительное снижение трудозатрат рекрутеров на первичный скрининг, повышение релевантности рекомендаций кандидатов, удобная работа с данными на русском и других языках, получение прозрачного инструмента с объяснимыми результатами поиска по цене существенно ниже коммерческих аналогов. Для исполнителя результатом проекта выступает создание работоспособного прототипа рыночного продукта, демонстрация навыков работы с современными технологиями искусственного интеллекта и получение практического опыта разработки масштабируемых решений для сферы HR-технологий.

7.2 Расчет инвестиций в разработку программного средства для его реализации на рынке

При разработке программного продукта учитываются несколько основных направлений финансирования: оплата труда разработчиков, дополнительные выплаты, социальные обязательства, сопутствующие расходы, расходы на реализацию. Подробные расчёты по каждому пункту представлены далее.

7.2.1 Оплата труда разработчиков определяется исходя из численности специалистов, их должностных окладов и трудоёмкости выполняемых задач. Для расчёта применяется выражение, записанное в формуле 7.1.

Расчёт осуществляется по формуле

$$Z_o = K_{\text{пр}} \sum_{i=1}^n Z_{\text{чи}} \times t_i, \quad (7.1)$$

где $K_{\text{пр}}$ – коэффициент премий;

n – категория исполнителей, которые заняты разработкой;

$Z_{\text{чи}}$ – часовой оклад исполнителя i -ой категории, р.;

t_i – трудоёмкость работ исполнителя i -ой категории, ч.

Трудоёмкость выполнения работ определена с учётом масштаба и функциональной сложности интеллектуальной системы подбора персонала на основе многоагентной архитектуры с использованием LangGraph и LangChain. Для AI-инженера принято значение 320 часов, что отражает объём задач по проектированию и реализации оркестрации агентов, интеграции цепочек обработки данных, работе с семантическим поиском. Данный специалист отвечает за архитектуру, корректность бизнес-логики подбора кандидатов и интеграцию всех компонентов, что требует значительных временных.

Фронтенд-разработчику отведено 140 часов, поскольку реализация удобного пользовательского интерфейса для ввода запросов на естественном языке, визуализации ранжированных результатов с пояснениями релевантности и тепловыми картами совпадений является критически важным этапом для принятия системы пользователями. Кроме того, в его обязанности входит оптимизация UI/UX-элементов, адаптация под разные сценарии работы рекрутера и обеспечение интуитивной навигации, что также увеличивает трудоёмкость.

Тестировщику предусмотрено 100 часов. Этот объём времени необходим для подготовки тест-кейсов на разнообразные типы резюме и запросов, проведения функциональных, интеграционных и нагрузочных испытаний, включая проверку точности семантического поиска и стабильности агентов, а также документирования обнаруженных дефектов и контроля их исправления. Работа тестировщика обеспечивает надёжность системы, минимизируя риски некорректных рекомендаций кандидатов и сбоев при обработке реальных объёмов данных.

Дизайнеру выделено 80 часов, что связано с созданием интерактивных прототипов интерфейса, разработкой макетов экранов поиска, результатов и загрузки данных, а также проработкой визуальных решений для отображения степени соответствия кандидатов вакансии. Его задачи включают обеспечение удобства пользовательских сценариев, логичной навигации и визуальной целостности продукта, соответствующей современным требованиям к эргономике и эстетике HR-инструментов на базе искусственного интеллекта.

Затраты на основную заработную плату представлены в таблице 7.1.

Таблица 7.1 – Затраты на основную заработную плату

Категория исполнителя	Месячный оклад, р.	Часовой оклад, р.	Трудоемкость работ, ч	Итого, р.
AI-инженер	4 500	26,79	320	8 571,43
Фронтенд разработчик	2 400	14,29	140	2 000,00
Тестировщик	2 900	17,26	100	1 726,19
Дизайнер	2 200	13,09	80	1 047,62
Итого				13 345,24
Премия (55%) *				7 339,88
Всего затраты на основную заработную плату сотрудников				20 685,12

7.2.2 Формирование затрат на дополнительную заработную плату разработчиков. Дополнительная заработная плата представляет 10% от основной. В формуле 7.2 приведён расчет дополнительной заработной платы.

$$З_д = \frac{З_о \cdot Н_д}{100}, \quad (7.2)$$

где $З_о$ – основная заработная плата, р.;
 $Н_д$ – норматив дополнительной заработной платы, %.

7.2.3 Формирование отчислений на социальные нужды. По состоянию на март 2026г., норматив отчислений в ФСЗН и Белгосстрах представляет 34,6%. Учтём это значение и рассчитаем итоговые отчисления исходя из основной и дополнительной заработной платы. Вычисления представлены в формуле 7.3.

$$Р_{соц} = \frac{(З_о + З_д) \cdot Н_{соц}}{100}, \quad (7.3)$$

где $Н_{соц}$ – норматив социальных отчислений, %.

7.2.4 Формирование прочих расходов. Затраты на прочие расходы формируются из основной заработной платы и норматива 30% от заработной платы. Вычисления представлены в формуле 7.4.

$$Р_{пр} = \frac{З_о \cdot Н_{пр}}{100}, \quad (7.4)$$

где $Н_{пр}$ – норматив прочих расходов, %.

7.2.5 Формирование расходов на реализацию. Затраты на расходы на реализацию формируются из основной заработной платы и норматива 3% от заработной платы. Вычисления представлены в формуле 7.5.

$$P_p = \frac{Z_o \cdot H_p}{100}, \quad (7.5)$$

где H_p – норматив расходов на реализацию, %.

7.2.6 Формирование общей суммы затрат. В их состав входят расходы на оплату труда основной команды разработчиков, дополнительные выплаты, обязательные социальные отчисления, прочие расходы и расходы на реализацию. Общая сумма затрат определялась по формуле 7.6.

$$Z_p = Z_o + Z_d + P_{\text{соц}} + P_{\text{пр}} + P_p. \quad (7.6)$$

Таким образом, величина затрат на разработку программного средства указана в таблице 7.2.

Таблица 7.2 – Затраты на разработку программного средства

Название статьи затрат	Формула/таблица для расчета	Значение, р.
Основная заработная плата разработчиков	Таблица 1.1	20 685,12
Дополнительная заработная плата разработчиков	$Z_d = \frac{20\,685,12 \cdot 10}{100}$	2 068,51
Отчисление на социальные нужды	$P_{\text{соц}} = \frac{(20\,685,12 + 2\,068,51) \cdot 34,6}{100}$	7 872,76
Прочие расходы	$P_{\text{пр}} = \frac{20\,685,12 \cdot 30}{100}$	6 205,54
Расходы на реализацию	$P_p = \frac{20\,685,12 \cdot 3}{100}$	620,55
Общая сумма затрат на разработку	$Z_p = 20\,685,12 + 2\,068,51 + 7\,872,76 + 6\,205,54 + 620,55$	37 452,48

Общая сумма затрат составила 37 452,48 белорусских рублей.

7.3 Расчет экономического эффекта от реализации программного средства на рынке

Оценка экономического эффекта от реализации программного средства позволит определить его коммерческую привлекательность и потенциальную

прибыльность. Эта величина зависит от прогнозируемого объема продаж, в данном случае количества подписок, стоимости продукта и затрат на его разработку.

Для определения цены программного средства проведен анализ аналогичных решений на рынке в 2026 году. Большинство современных HR-платформ с такими элементами ИИ, как семантический поиск, анализ резюме, гибридный поиск, автоматизация скрининга, работают по модели SaaS-подписки (ежемесячная либо ежегодная оплата за пользователя или за компанию). В таблице 7.3 приведены примеры аналогичных продуктов и их стоимость за использование.

Таблица 7.3 – Стоимость использования продуктов-аналогов

Название продукта	Модель подписки	Цена, \$ за год
Talantix	Подписка за рекрутера на год	600
Huntflow	Подписка за рекрутера на год	950
Поток Рекрутмент	Подписка за рекрутера на год	815
Garmony.ai	Подписка за компанию на месяц	1 260
FriendWork	Подписка за компанию на месяц	650

Из представленной таблицы можно вычислить, что средняя стоимость подписки на год составляет 854,4 доллара США, что в пересчете на белорусские рубли составляет 2500 рублей.

Для расчета прироста чистой прибыли необходимо учесть налог на добавленную стоимость (НДС). Отпускная цена подписки на программное средство установлена на уровне 1900 рублей за одного рекрутера или малую компанию. Ожидается, что за год количество покупок подписки составит примерно 5. Ставка налога на добавленную стоимость, действующая на март 2026 года согласно законодательству Республики Беларусь, составляет 20%. Налог на добавленную стоимость рассчитывается по формуле:

$$\text{НДС} = \frac{\text{Ц}_{\text{отп}} \cdot N \cdot \text{Н}_{\text{д.с}}}{100 + \text{Н}_{\text{д.с}}}, \quad (7.7)$$

где N – количество оплат за подписку на программный продукт за год, шт.;
 $\text{Ц}_{\text{отп}}$ – отпускная цена подписки программного средства, р.;
 $\text{Н}_{\text{д.с}}$ – ставка налога на добавленную стоимость, %.
Тогда налог на добавленную стоимость составляет:

$$\text{НДС} = \frac{1\,900 \cdot 5 \cdot 20}{100 + 20} = 1\,583,33 \text{ р.} \quad (7.8)$$

После вычисления налога на добавленную стоимость, можно рассчитать прирост чистой прибыли, который разработчик получит от

реализации программного продукта. Ставка налога на прибыль, в соответствии с действующим законодательством на март 2026 года, составляет 25%. Рентабельность продаж программного продукта установлена на уровне 20%. Формула для вычисления прироста чистой прибыли:

$$\Delta\Pi_{\text{ч}}^{\text{p}} = (\Pi_{\text{отп}} \cdot N - \text{НДС}) \cdot R_{\text{пр}} \cdot (1 - \frac{H_{\text{п}}}{100}), \quad (7.9)$$

где $\Pi_{\text{отп}}$ – отпускная цена подписки программного средства, р.;
 N – количество оплат за подписку на программный продукт за год, шт.;
 НДС – сумма налога на добавленную стоимость, р.;
 $R_{\text{пр}}$ – рентабельность продаж;
 $H_{\text{п}}$ – ставка налога на прибыль.
 Тогда прирост чистой прибыли составляет:

$$\Delta\Pi_{\text{ч}}^{\text{p}} = (1\,900 \cdot 5 - 1\,583,33) \cdot 20 \cdot (1 - \frac{25}{100}) = 118\,750,05 \text{ р.} \quad (7.10)$$

7.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке

Для оценки экономической эффективности разработки и внедрения программного продукта на рынок, необходимо провести сравнительный анализ затрат на его разработку и ожидаемого прироста чистой прибыли за год. Если сумма затрат на разработку меньше годового экономического эффекта, то можно заключить, что инвестиции окупятся менее чем за один год.

Оценка эффективности инвестиций проводится с использованием расчета рентабельности инвестиций (англ. Return on Investment, ROI). Формула для расчета рентабельности инвестиций:

$$ROI = \frac{\Delta\Pi_{\text{ч}}^{\text{p}} - Z_{\text{р}}}{Z_{\text{р}}} \cdot 100\%, \quad (7.11)$$

где $\Delta\Pi_{\text{ч}}^{\text{p}}$ – прирост чистой прибыли, полученной от реализации программного средства на рынке, р;
 $Z_{\text{р}}$ – затраты на разработку и реализацию программного средства, р.
 Тогда рентабельность инвестиций составляет:

$$ROI = \frac{118\,750,05 - 37\,452,48}{37\,452,48} \cdot 100\% = 217,07\%. \quad (7.12)$$

Данный уровень рентабельности подтверждает экономическую обоснованность реализации проекта и свидетельствует о его высоком потенциале для будущего роста и развития.

7.5 Вывод об экономической целесообразности реализации проектного решения

Проект по разработке интеллектуальную систему подбора персонала с семантическим анализом агрегированных данных продемонстрировал положительные результаты с точки зрения экономической целесообразности. Ожидаемый ROI составляет 217,07%, что указывает на высокую рентабельность и привлекательность инвестиций. Чистая прибыль, которую проект может генерировать ежегодно, составляет около 119 тыс. белорусских рублей. Эти показатели подтверждают, что проект обладает значительным потенциалом для получения прибыли в краткосрочной и среднесрочной перспективе.

Для привлечения большего числа пользователей, проект может предложить базовую бесплатную версию программного средства с ощутимым ограничением ресурсов (количество резюме и вакансий). Такой подход позволит привлечь малый и средний бизнес, дать возможность протестировать систему на реальных задачах и плавно перевести на платную подписку.

Чтобы выделиться и обеспечить конкурентоспособность, продукт должен сохранять и усиливать ключевые преимущества: существенно более низкую цену, полную прозрачность алгоритмов, встроенную мультиязычность для рынков СНГ и экспорта.

Кроме того, для долгосрочной конкурентоспособности необходим дополнительный функционал: интеграции с ведущими порталами (hh.ru, SuperJob), расширенная аналитика эффективности, инструменты снижения предвзятости, мобильное приложение для рекрутеров и кандидатов, а также готовые шаблоны для разных отраслей.

Основные риски связаны с необходимостью постоянного улучшения качества сопоставления, обеспечения скорости и надёжности при росте нагрузки, а также строгого соблюдения требований к защите персональных данных. При грамотном маркетинге и фокусе на уникальных сильных сторонах проект имеет все шансы выйти на самоокупаемость и занять устойчивую нишу в сегменте доступных ИИ-решений для подбора персонала.

ЗАКЛЮЧЕНИЕ

Должно быть минимум на 81 странице!!

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Каплан, А. Искусственный интеллект, цифровой мир и трансформация рынка труда / А. Каплан, М. Хенляйн // Цифровая экономика. – 2023. – № 4. – С. 12–25.
- [2] Мишра, С. Ограничения традиционных систем поиска персонала на основе ключевых слов / С. Мишра, Р. Гупта // Journal of HR Technology. – 2022. – Vol. 15. – P. 45–58.
- [3] Вульдрих, М. Многоагентные системы: введение / М. Вульдрих, Г. Вейсс // Многоагентные системы. – 2021. – С. 3–20.
- [4] Brown, T. Language Models are Few-Shot Learners [Электронный ресурс] / T. Brown [et al.] // arXiv preprint arXiv:2005.14165. – 2020. – Режим доступа: <https://arxiv.org/abs/2005.14165> – Дата доступа: 15.02.2026.
- [5] Vaswani, A. Attention Is All You Need [Электронный ресурс] / A. Vaswani [et al.] // Advances in Neural Information Processing Systems. – 2017. – P. 5998–6008. – Режим доступа: <https://arxiv.org/abs/1706.03762> – Дата доступа: 12.02.2026.
- [6] Shazeer, N. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer [Электронный ресурс] / N. Shazeer [et al.] // arXiv preprint arXiv:1701.06538. – 2017. – Режим доступа: <https://arxiv.org/abs/1701.06538> – Дата доступа: 18.02.2026.
- [7] Yang, A. Qwen Technical Report [Электронный ресурс] / A. Yang [et al.] // arXiv preprint arXiv:2309.16609. – 2023. – Режим доступа: <https://arxiv.org/abs/2309.16609> – Дата доступа: 20.02.2026.
- [8] Chase, H. LangChain [Электронный ресурс] / H. Chase. – Электронные данные. – Режим доступа: <https://github.com/langchain-ai/langchain> – Дата доступа: 22.02.2026.
- [9] LangChain AI. LangGraph Documentation [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://langchain-ai.github.io/langgraph/> – Дата доступа: 25.02.2026.
- [10] Lewis, P. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [Электронный ресурс] / P. Lewis [et al.] // Advances in Neural Information Processing Systems. – 2020. – Vol. 33. – P. 9459–9474. – Режим доступа: <https://arxiv.org/abs/2005.11401> – Дата доступа: 27.02.2026.
- [11] Gao, Y. Retrieval-Augmented Generation for Large Language Models: A Survey [Электронный ресурс] / Y. Gao [et al.] // arXiv preprint arXiv:2312.10997. – 2023. – Режим доступа: <https://arxiv.org/abs/2312.10997> – Дата доступа: 01.03.2026.
- [12] Mikolov, T. Distributed Representations of Words and Phrases and their Compositionality [Электронный ресурс] / T. Mikolov [et al.] // Advances in Neural Information Processing Systems. – 2013. – P. 3111–3119. – Режим доступа: <https://arxiv.org/abs/1310.4546> – Дата доступа: 03.03.2026.
- [13] Devlin, J. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding [Электронный ресурс] / J. Devlin [et al.] // arXiv preprint

arXiv:1810.04805. – 2018. – Режим доступа: <https://arxiv.org/abs/1810.04805> – Дата доступа: 05.03.2026.

[14] Muennighoff, N. MTEB: Massive Text Embedding Benchmark [Электронный ресурс] / N. Muennighoff [et al.] // arXiv preprint arXiv:2210.07316. – 2022. – Режим доступа: <https://arxiv.org/abs/2210.07316> – Дата доступа: 06.03.2026.

[15] Malkov, Y. A. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs / Y. A. Malkov, D. A. Yashunin // IEEE Transactions on Pattern Analysis and Machine Intelligence. – 2018. – Vol. 42, № 4. – P. 824–836.

[16] Qdrant Team. Qdrant Vector Database Documentation [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://qdrant.tech/documentation/> – Дата доступа: 10.02.2026.

[17] Resume Matcher Project [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://github.com/srbhr/resume-matcher> – Дата доступа: 14.02.2026.

[18] Eightfold AI. Talent Intelligence Platform [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://www.eightfold.ai/> – Дата доступа: 17.02.2026.

[19] Phenom People. Talent Experience Management [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://www.phenompeople.com/> – Дата доступа: 21.02.2026.

[20] pdfplumber Documentation [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://github.com/jsvine/pdfplumber> – Дата доступа: 24.02.2026.

[21] Xu, Y. LayoutLM: Pre-training of Text and Layout for Document Image Understanding [Электронный ресурс] / Y. Xu [et al.] // Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. – 2020. – P. 1192–1201. – Режим доступа: <https://arxiv.org/abs/1912.13318> – Дата доступа: 28.02.2026.

[22] IBM Research. Docling: Document Understanding Toolkit [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://github.com/DS4SD/docling> – Дата доступа: 02.03.2026.

[23] Unstructured IO. Unstructured Open-Source Library [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://unstructured.io/> – Дата доступа: 04.03.2026.

Вычислительные машины, системы и сети: дипломное проектирование [Электронный ресурс]. – Электронные данные. – Режим доступа: https://www.bsuir.by/m/12_100229_1_136308.pdf – Дата доступа: 31.03.2026.

Экономика проектных решений: методические указания по экономическому обоснованию дипломных проектов [Электронный ресурс]. – Электронные данные. – Режим доступа: https://www.bsuir.by/m/12_100229_1_161144.pdf – Дата доступа: 31.03.2026.

ПРИЛОЖЕНИЕ А

(обязательное)

Исходный код программы

Код начинается с этого места;

001 Можно с нумерацией строк кода;

002 Шрифт может быть от 12 до 8 pt

ПРИЛОЖЕНИЕ Б
(обязательное)

Название приложения

ПРИЛОЖЕНИЕ В
(обязательное)

Отчет о проверке на заимствования

ПРИЛОЖЕНИЕ Г
(обязательное)

Спецификация

ПРИЛОЖЕНИЕ Д
(обязательное)

Ведомость документов